

Multi-Chain Payment Gateway

Project Completion & Technical Documentation Report

Project Type: Web3 / Blockchain Payment Infrastructure

Primary Tested Network: Ethereum Sepolia

Frontend Deployment: Vercel

Backend Deployment: Render

Database: PostgreSQL

Repository: github.com/max-wire/multi-chain-payment-gateway

Technical documentation • MVP deployment record • September 2026

1. Executive Summary

- The Multi-Chain Payment Gateway is a non-custodial Web3 payment infrastructure application designed to allow merchants to create cryptocurrency payment requests and customers to complete those payments directly from their blockchain wallets.
- The system connects a React-based merchant interface with a Node.js backend, PostgreSQL database, and blockchain smart contract. Payments are not considered confirmed based solely on frontend information. Instead, the backend independently verifies blockchain transactions before recording them as confirmed payments.
- The architecture is designed for multiple EVM-compatible networks. Ethereum Sepolia was selected as the active and tested network for the MVP, while Ethereum mainnet, BNB Chain, Polygon, and SecureChain are represented through the network configuration architecture and can be activated when their corresponding RPC and contract configurations are available.

2. Project Objectives

- Build a Web3 payment gateway for cryptocurrency payments.
- Allow merchants to create payment requests.
- Generate unique payment identifiers.
- Allow customers to connect blockchain wallets.
- Provide payment links that customers can open directly.
- Submit payments through blockchain transactions.
- Independently verify transactions on the backend.
- Track transaction confirmations.
- Protect against duplicate transaction verification.
- Provide merchant payment history.
- Provide invoice management.
- Provide a merchant dashboard.
- Provide basic network administration visibility.
- Establish an architecture capable of supporting multiple EVM networks.

3. System Architecture

Frontend

- React
- TypeScript
- Vite
- Tailwind CSS
- React Router
- Wagmi
- Viem

Backend

- Node.js

- Express
- TypeScript
- Zod
- Prisma
- ethers.js

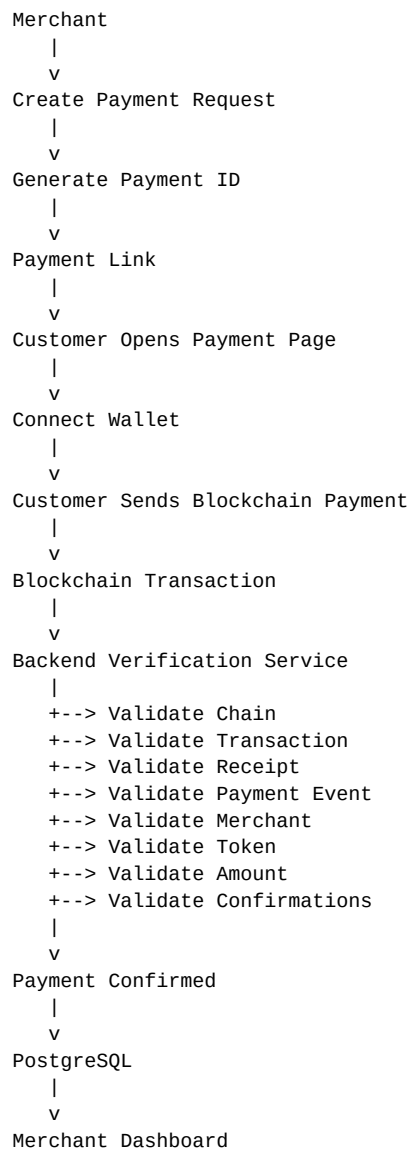
Database

- PostgreSQL is used to persist merchants, payments, transactions, invoices, and token/network-related configuration.

Blockchain

- The blockchain layer consists of Solidity smart contracts deployed/tested using Ethereum Sepolia. The system uses blockchain events and transaction receipts as part of the payment verification process.

4. High-Level Payment Architecture



- The architecture separates the customer-facing interface from the server-side verification process.

5. Payment Creation

- Merchants can create payment requests through the frontend.
- Each payment contains a unique payment ID, merchant, amount, token/currency, description, expiration information, and payment status.
- A payment link can then be shared with a customer. The customer does not need access to the merchant's private keys.

6. Customer Wallet Integration

- The frontend supports browser-injected wallets such as MetaMask.
- The wallet connection is handled through Wagmi and Viem.
- The connected wallet address is used as the customer identity for the blockchain payment transaction.
- The application does not request or store customer private keys or seed phrases.

7. Blockchain Payment

- The PaymentGateway smart contract supports native ETH payments, ERC-20 token payments, unique payment IDs, merchant recipient addresses, payment status tracking, and payment events.
- The currently tested production flow uses native ETH on Ethereum Sepolia.
- The ERC-20 functionality exists in the smart contract architecture but was not used as the primary tested payment flow for this MVP.

8. Server-Side Transaction Verification

- The backend does not simply trust a frontend response saying that a payment succeeded.
- Verification checks the payment ID, blockchain network/chain ID, transaction hash, transaction receipt, transaction success status, PaymentReceived event, merchant address, token address, payment amount, required confirmations, and duplicate transaction protection.
- Only after these checks are satisfied can the payment be recorded as confirmed.

9. Duplicate Transaction Protection

- The database uses the combination of chainId + transactionHash to uniquely identify blockchain transactions.
- This prevents the same blockchain transaction from being recorded multiple times for the same network.

10. Confirmation Tracking

- The backend tracks blockchain confirmations before treating a transaction as sufficiently confirmed.
- This provides additional protection against immediately accepting a transaction before the required confirmation threshold has been reached.
- The system distinguishes between a payment that has merely been created and a payment that has been successfully confirmed on-chain.

11. Merchant Dashboard

- The merchant dashboard provides an overview of payment activity, including merchant information, total payments, confirmed payments, pending/created payments, merchant wallet, and merchant ID.
- The production dashboard was successfully tested against the deployed backend.

12. Transaction History

- The transaction interface provides merchants with payment history.
- The interface displays payment ID, blockchain transaction, amount, payment status, description, and creation date.
- Confirmed blockchain transactions can include a link to the corresponding blockchain explorer transaction.

13. Invoice Management

- Invoices support invoice numbers, customer information, amount, currency, description, due dates, invoice status, and association with payments.
- Supported invoice states include ISSUED, PAID, OVERDUE, and CANCELLED.

14. Basic Administration Panel

- The MVP includes a read-only administration panel available at /admin.
- The panel provides visibility into configured networks, including network name, chain ID, native asset, payment-enabled status, testnet status, explorer URL, and ERC-20 capability.
- Network activation and token configuration remain configuration-based in the current MVP.
- Administrative authentication and authorization are planned for a future version.

15. Supported Network Architecture

- Ethereum (Chain ID 1) — Configured / inactive
- Ethereum Sepolia (Chain ID 11155111) — Active and tested
- BNB Chain (Chain ID 56) — Configured / inactive
- Polygon (Chain ID 137) — Configured / inactive
- SecureChain (Chain ID 34) — Configured / inactive
- Ethereum Sepolia is the active and tested payment network for this MVP. The other networks are represented in the architecture but have not been presented as fully deployed payment environments.

16. Security Measures

- Non-custodial wallet architecture
- No storage of customer private keys
- Server-side blockchain verification
- Chain ID validation
- Transaction receipt validation
- Transaction status validation
- Payment event validation
- Merchant validation

- Token validation
- Amount validation
- Confirmation validation
- Duplicate transaction protection
- Zod request validation
- Helmet security middleware
- Environment variables for sensitive configuration
- Sensitive credentials and private keys are intended to remain outside source control.

17. Testing and Validation

- Application validation covered payment creation, wallet connection, payment links, customer payment page, merchant dashboard, transaction history, invoice functionality, and the basic administration panel.
- Ethereum Sepolia was used for live test transactions.
- Blockchain validation covered valid transactions, payment IDs, merchant addresses, payment amounts, transaction receipts, confirmation counts, duplicate transaction scenarios, and invalid transaction conditions.
- A confirmed Sepolia payment was successfully displayed in the production transaction history.

18. Production Deployment

- Frontend — Platform: Vercel — Production URL: <https://frontend-ten-ecru-19.vercel.app/>
- Backend — Platform: Render — Production URL: <https://multi-chain-payment-gateway.onrender.com/>
- Database — PostgreSQL
- Blockchain — Ethereum Sepolia
- The production frontend communicates with the deployed backend API.
- The production dashboard was successfully tested after resolving a merchant-record uniqueness issue in the production database.

19. Deployment Issue Resolved

- During production verification, the dashboard initially returned an internal server error when attempting to create a merchant.
- The backend logs identified the cause as a PostgreSQL unique constraint violation on the merchant email field.
- The merchant creation service was updated to reuse an existing merchant record when an appropriate existing wallet or email record is found rather than attempting to create a duplicate record.
- The fix was committed and deployed successfully.
- This demonstrated the importance of validating the complete production path rather than relying only on local development testing.

20. Project Structure

```
multi-chain-payment-gateway/
├── backend/
│   ├── prisma/
│   └── src/
└── contracts/
```

```
■■■ frontend/
■   ■■■ src/
■■■ scripts/
■■■ test/
■■■ hardhat.config.ts
■■■ package.json
■■■ README.md
```

21. Local Development

Frontend

```
cd frontend
npm install
npm run dev
```

- Development server: <http://localhost:5173>

Backend

```
cd backend
npm install
npm run dev
```

- Development server: <http://localhost:3000>

Health Check

```
GET /health
```

22. Build Validation

Frontend production build

```
cd frontend
npm run build
```

Backend type checking

```
cd backend
npm run typecheck
```

Backend production build

```
cd backend
npm run build
```

- The backend production build was successfully completed using the TypeScript compiler.

23. Current Project Status

Completed

- Payment creation
- Unique payment IDs
- Wallet connection
- Customer payment page

- Payment links
- Native ETH payment flow
- Server-side transaction verification
- Confirmation tracking
- Duplicate transaction protection
- Transaction history
- Invoice system
- Merchant dashboard
- Basic administration panel
- Security validation
- Ethereum Sepolia testing
- Frontend deployment
- Backend deployment
- Production API communication
- Project README documentation

Configured but Inactive

- Ethereum mainnet
- BNB Chain
- Polygon
- SecureChain
- These networks require their respective RPC endpoints, gateway deployments, and network-specific configuration before being activated for payment processing.

24. Limitations

- Ethereum Sepolia is the only actively tested payment network.
- ERC-20 payment capability exists in the smart contract but was not the primary tested payment flow.
- The administration panel is read-only.
- Admin authentication and authorization are not implemented.
- Rate limiting has not been implemented.
- Multiple dedicated wallet connectors have not been implemented.
- Automated production monitoring has not been implemented.
- Fiat settlement is not implemented.
- Subscription payments are not implemented.
- These limitations define the boundary between the current MVP and a future production-scale payment platform.

25. Future Development

- Activate BNB Chain.
- Activate Polygon.

- Activate SecureChain.
- Configure production ERC-20 tokens.
- Add additional wallet connectors.
- Add authenticated administration.
- Add role-based access control.
- Add API rate limiting.
- Add automated monitoring and alerting.
- Add advanced payment analytics.
- Add subscription payments.
- Add fiat settlement capabilities.

26. Conclusion

- The Multi-Chain Payment Gateway successfully demonstrates an end-to-end Web3 payment workflow.
- The completed MVP connects a merchant interface, customer wallet, blockchain smart contract, backend verification service, and PostgreSQL database into a single payment system.
- The most important architectural feature is the separation between payment initiation and payment verification. Blockchain transactions are independently inspected by the backend before being treated as confirmed payments.
- Ethereum Sepolia provides the currently active test environment, while the centralized network configuration provides a foundation for extending the gateway to additional EVM-compatible networks.
- The project has been built, tested, documented, and deployed with a clear path for future expansion.

27. Project Links

- GitHub Repository: <https://github.com/max-wire/multi-chain-payment-gateway>
- Live Frontend: <https://frontend-ten-ecru-19.vercel.app/>
- Production Backend: <https://multi-chain-payment-gateway.onrender.com/>
- Ethereum Sepolia Explorer: <https://sepolia.etherscan.io/>

28. Final Deliverable Summary

- React/TypeScript frontend
- Node.js/Express backend
- PostgreSQL database
- Solidity payment gateway
- Ethereum Sepolia deployment/testing
- Wallet integration
- Payment creation
- Payment links
- Server-side blockchain verification
- Transaction history
- Invoice management

- Merchant dashboard
- Basic administration panel
- Production frontend deployment
- Production backend deployment
- Technical documentation